

Q.(1) Why is main() method static?

Answer-- The main method is static because it can be called by the JVM before any objects of the class are created.

Q.(2) What is JVM? Why is Java called the "Platform Independent Programming Language"?

Answer-- JVM (Java Virtual Machine) is an abstract machine that provides a runtime environment in which Java bytecode can be executed. Java is platform independent because it is compiled into bytecode that can run on any platform with a JVM.

Q.(3) What is the Difference between JDK and JRE? -

Answer-- JDK (Java Development Kit) is a software development environment used for developing Java applications. JRE (Java Runtime Environment) is a software package that contains what is required to run a Java program.

Q.(4) What are the Data Types supported by Java? What is Autoboxing and Unboxing? -

Answer-- Java supports primitive data types (like int, char, float, etc.) and reference data types (like String, Array, etc.). Autoboxing is the automatic conversion of primitive types to their corresponding object wrapper classes. Unboxing is the reverse process.

Q.(5) What is the difference between equals() and == ? -

Answer-- The '==' operator checks if the two objects point to the same memory location whereas the equals() method checks if the two objects are equal in terms of their value.

Q.(6) What are different types of access modifiers in Java? -

Answer-- Java has four access modifiers: public, private, protected, and default.

Q.(7) What is the use of final keyword? -

Answer-- The final keyword in Java is used to restrict the user. It can be used in many contexts. Final can be:

variable: value is constant

method: can't be overridden

class: can't be inherited

Q.(8) What is volatile keyword? -

Answer-- The volatile keyword in Java is used to mark a Java variable as "being stored in main memory". More precisely that means, the value of this variable will never be cached thread-locally: all reads and writes will go straight to "main memory".

Q.(9) What is a transient variable? -

Answer-- A transient variable in Java is a variable that can't be serialized.

Q.(10) What is a static keyword? -

Answer-- The static keyword in Java is used for memory management mainly. It can be used in many contexts. A static variable is common to all the objects of the class because it is a class level variable.

Q.(11) What is immutable object? Can you write immutable object? -

Answer-- An immutable object is an object whose state cannot be changed after it is created. To make a class immutable, follow these rules:

Don't provide "setter" methods — methods that modify fields or objects referred to by fields.

Make all fields final and private.

Don't allow subclasses to override methods.

Q.(12) What is difference between stringbuffer and stringbuilder?

Answer-- StringBuffer is synchronized and StringBuilder is not. This makes StringBuffer thread-safe and StringBuilder not thread-safe.

Q.(13) What is String in Java? String is a data type? -

Answer-- String is a class in Java and not a primitive data type. It is used to represent a sequence of characters.

Q.(14) What are different ways to create String Object? -

Answer-- There are two ways to create a String object: using the new keyword and using a string literal.

Q.(15) What are the various access specifiers for Java classes? -

Answer-- Java classes can have public or default access.

Q.(16) What are Loops in Java? What are three types of loops? -

Answer-- Loops are used to execute a block of code repeatedly. Java has three types of loops: for loop, while loop, and do-while loop.

Q.(17) What is ternary operator? -

Answer-- The ternary operator is a shorthand for the if-else statement. It takes three operands and evaluates to one of two values based on a condition.

Q.(18) Can a variable be local and static at the same time? -

Answer-- No, a variable cannot be both local and static at the same time.

Q.(19) If an application has multiple classes in it, is it okay to have a main method in more than one class? -

Answer-- Yes, it is okay to have a main method in more than one class. However, the JVM starts the execution from the class that is specified as the main class.

Q.(20) What is the purpose of garbage collection in Java, and when is it used? -

Answer-- Garbage collection is the process of freeing up memory by removing objects that are no longer in use. It is used automatically by the JVM.

Q.(21) What are the different ways to call garbage collection? -

Answer-- Garbage collection can be called using the `System.gc()` method or by calling `Runtime.getRuntime().gc()`.

Q.(22) When is the `finalize()` called? What is the purpose of finalization? -

Answer-- The `finalize()` method is called by the JVM just before an object is garbage collected. Its purpose is to perform cleanup

processing.

Q.(23) When does an Object becomes eligible for Garbage collection in Java? -

Answer-- An object becomes eligible for garbage collection in Java when it is no longer reachable.

Q.(24) What are different oops concept in java? -

Answer-- OOPS concepts in Java are:

Abstraction

Encapsulation

Inheritance

Polymorphism

Q.(25) What is Encapsulation? -

Answer-- Encapsulation is the mechanism of hiding the internal details or mechanics of how an object does something.

Q.(26) What is polymorphism? -

Answer-- Polymorphism is the ability of an object to take on many forms. The most common use of polymorphism in OOP occurs when a

parent class reference is used to refer to a child class object.

Q.(27) What is inheritance? -

Answer-- Inheritance is a process where one class acquires the properties (methods and fields) of another.

Q.(28) What is an abstraction? -

Answer-- Abstraction is a process of hiding the implementation details and showing only functionality to the user.

Q.(29) Does java support multiple inheritance? Why? -

Answer-- Java does not support multiple inheritance of classes to avoid the diamond problem. However, it supports multiple inheritance of interfaces.

Q.(30) What is difference between abstract class and interface?

Answer-- An abstract class is a class that contains both abstract and regular methods. An interface is a reference type in Java, similar to a class, that can contain only constants, method signatures, default methods, static methods, and nested types.

Q.(31) What is difference between Runtime and compile time

polymorphism? -

Answer-- Runtime polymorphism is also known as dynamic method dispatch and is resolved using method overriding. Compile-time polymorphism is achieved using method overloading.

Q.(32) What is difference between overloading and overriding? -

Answer-- Overloading is when two or more methods in the same class have the same name but different parameters. Overriding is when a subclass provides a specific implementation for a method that is already provided by its parent class.

Q.(33) Explain with example to describe when to use abstract class and interface? -

Answer-- Use an abstract class when you want to provide a common base for a family of related classes, but you want to force subclasses to provide their own implementation for some methods. Use an interface when you want to specify a behavior that a family of classes must implement.

Q.(34) Can an abstract class have a static method? -

Answer-- Yes, an abstract class can have a static method.

Q.(35) Can an abstract class have a constructor? -

Answer-- Yes, an abstract class can have a constructor.

Q.(36) Java supports pass by value or pass by reference? -

Answer- Java is always pass by value.

Q.(37) What do you mean by Constructor? -

Answer-- A constructor in Java is a special method that is used to initialize objects.

Q.(38) When super keyword is used? -

Answer-- The super keyword in Java is used to refer to the immediate parent class object.

Q.(39) When this keyword is used? -

Answer-- The this keyword in Java is used to refer to the current object.

Q.(40) What is a singleton class? -

Answer-- A singleton class in Java is a class that allows only one object to be created.

Q.(41) What's the base class in Java from which all classes are derived?

Answer-- The base class in Java from which all classes are derived is Object.

Q.(42) Can we have two methods in a class with the same name?

Answer-- Yes, we can have two methods in a class with the same name as long as they have different parameter lists.

Q.(43) What is "IS-A" and "Has-A" relation? -

Answer-- "IS-A" is a relationship between a parent class and a child class. "Has-A" is a relationship between two classes where one class has an instance of another class.

Q.(44) What is Exception in Java? -

Answer-- An exception is an event that occurs during the execution of a program that disrupts the normal flow of the program's instructions.

Q.(45) What are the Exception Handling Keywords in Java?

Answer-- The exception handling keywords in Java are try, catch, finally, throw, and throws.

Q.(46) Explain Java Exception Hierarchy?

Answer-- Java exception hierarchy consists of the Throwable class and its subclasses. The Throwable class has two main subclasses: Error and Exception. The Exception class is further divided into two subclasses: RuntimeException and IOException.

Q.(47) Explain Java 7 Feature and multi-catch block?

Answer-- Java 7 introduced several features, one of them is the multi-catch block. Prior to Java 7, if a method contained multiple catch blocks to handle different exceptions, each catch block had to be written separately. However, with the introduction of multi-catch block, multiple exceptions can be caught using a single catch block. The exceptions should be related and should not have a common base class other than Exception.

Here's an example:

csharp

Verify

Edit

Full Screen

Copy code

```
try {
```

```
        // code that may throw exceptions
    } catch (IOException | SQLException e) {

        // handle exception
    }
```

In the above example, both `IOException` and `SQLException` can be caught using a single catch block.

Q.(48) What is difference between Checked and Unchecked Exception in Java?

Answer-- In Java, exceptions can be classified into two types: checked and unchecked exceptions.

Checked exceptions are exceptions that are checked at compile-time. These exceptions need to be either caught or declared using the `throws` keyword in the method signature. Examples of checked exceptions are `IOException`, `SQLException`, etc.

Unchecked exceptions are exceptions that are not checked at compile-time. These exceptions are not required to be caught or declared. Examples of unchecked exceptions are `NullPointerException`, `ArrayIndexOutOfBoundsException`, etc.

Q.(49) What is difference between throw and throws keyword in Java?

Answer-- The throw keyword is used to throw an exception explicitly within a method. It is followed by an instance of a throwable class.

The throws keyword is used in the method signature to declare an exception that the method may throw. It is followed by the name of the exception class.

Here's an example:

java

Verify

Edit

Full Screen

Copy code

```
public void myMethod() throws IOException {  
    throw new IOException();  
}
```

In the above example, myMethod declares that it may throw an IOException using the throws keyword. Inside the method, an IOException is thrown using the throw keyword.

Q.(50) What is the difference between Exception and Error in java?

Answer-- In Java, both Exception and Error are subclasses of Throwable. However, they are used to indicate different types of

conditions.

Exceptions are used to indicate exceptional conditions that can be handled by the application. Exceptions can be caught and handled using try-catch blocks.

Errors are used to indicate critical conditions that cannot be handled by the application. Errors usually indicate problems with the runtime environment, such as `OutOfMemoryError`, `StackOverflowError`, etc.

Q.(51) What is the importance of finally block in exception handling?

Answer-- The finally block is used to execute a block of code that should be executed whether an exception is thrown or not. The finally block is executed after the try and catch blocks have been executed. The finally block is useful for cleaning up resources, such as closing database connections, files, etc.

Q.(52) How to write custom exception in Java?

Answer-- To write a custom exception in Java, you need to create a new class that extends the `Exception` class or one of its subclasses. Here's an example:

java

Verify

Edit

Full Screen

Copy code

```
public class CustomException extends Exception {  
    public CustomException(String message) {  
        super(message);  
    }  
}
```

In the above example, CustomException is a new exception class that extends the Exception class. It has a single constructor that takes a string message.

Q.(53) What is difference between final, finally and finalize in Java?

Answer-- final is a keyword used to denote that a variable, method, or class cannot be modified.

finally is a block of code that is executed after the try and catch blocks have been executed.

finalize is a method that is called by the garbage collector before an object is garbage collected.

Q.(54) What happens when exception is thrown by main method?

Answer-- If an exception is thrown by the main method, the JVM terminates the program and prints the stack trace of the exception.

Q.(55) Can we have an empty catch block?

Answer-- No, it is not recommended to have an empty catch block. If an exception is caught, it should be handled properly. Having an empty catch block can make debugging difficult.

Q.(56) What are the basic interfaces of Java Collections Framework?

Answer-- The basic interfaces of the Java Collections Framework are Collection, List, Set, and Map.

Q.(57) What is the benefit of Generics in Collections Framework?

Answer-- Generics provide type safety in the Java Collections Framework. With generics, you can specify the type of elements that a collection can contain. This helps prevent type casting errors and improves code readability.

Q.(58) How can an ArrayList be synchronized without using Vector?

Answer-- An ArrayList can be synchronized without using Vector by creating a synchronized wrapper around it using the `Collections.synchronizedList()` method.

Here's an example:

java

Verify

Edit

Full Screen

Copy code

```
List<String> list = new ArrayList<String>();
```

```
List<String> synchronizedList = Collections.synchronizedList(list);
```

In the above example, `synchronizedList` is a synchronized wrapper around the `ArrayList` list.

Q.(59) What is difference between an ArrayList and a vector?

Answer-- `ArrayList` is a resizable array that implements the `List` interface. It is not thread-safe.

`Vector` is a legacy class that implements the `List` interface. It is similar to `ArrayList`, but it is thread-safe.

Q.(60) What is difference between a HashMap and a Hashtable?

Answer-- `HashMap` is a hash table that implements the `Map` interface.

It is not thread-safe.

Hashtable is a legacy class that implements the Map interface. It is similar to HashMap, but it is thread-safe.

Q.(61) What are the classes implementing the List interface?

Answer-- The classes that implement the List interface are ArrayList, LinkedList, Vector, Stack, and CopyOnWriteArrayList.

Q.(62) Which all classes implement Set interface?

Answer-- The classes that implement the Set interface are HashSet, LinkedHashSet, TreeSet, CopyOnWriteArraySet, and EnumSet.

Q.(63) What is difference between List and a Set?

Answer-- List is an ordered collection of elements that can contain duplicates.

Set is an unordered collection of unique elements.

Q.(64) What is difference between Arrays and ArrayList?

Answer-- Arrays is a class that provides static methods for manipulating arrays.

ArrayList is a resizable array that implements the List interface.

Q.(65) When to use ArrayList or LinkedList?

Answer-- Use ArrayList when you need to access elements by index frequently.

Use LinkedList when you need to add or remove elements frequently.

Q.(66) What is the difference between iterator access and index access?

Answer-- Iterator access allows you to traverse a collection using an iterator.

Index access allows you to access elements by their index.

Q.(67) What is different between Iterator and ListIterator?

Answer-- Iterator is an interface that allows you to traverse a collection in one direction.

ListIterator is an interface that extends Iterator and allows you to traverse a List in both directions.

Q. (68) Can a null element be added to a TreeSet or HashSet?

Answer-- No, a NullPointerException is thrown if you try to add a null element to a TreeSet or HashSet.

Q.(69) Describe different states of a thread.

Answer-- The different states of a thread are:

New: A thread is in the new state when it is created but not started.

Runnable: A thread is in the runnable state when it is started and ready to run.

Running: A thread is in the running state when it is currently executing.

Blocked: A thread is in the blocked state when it is waiting for a resource or a synchronization object.

Terminated: A thread is in the terminated state when it has completed execution or has been stopped.

Q.(70) What is the difference between HashSet and TreeSet?

Answer-- HashSet is a hash table that implements the Set interface. It is not ordered and does not maintain any insertion order.

TreeSet is a sorted set that implements theSet interface. It is ordered and maintains the insertion order.

Q.(71) What is Collections Class?

Answer-- The Collections class is a utility class that provides static methods for manipulating collections. It provides methods for sorting, searching, and synchronizing collections.

Q.(72) What is the difference between processes and threads?

Answer-- A process is an independent program that runs in its own memory space.

A thread is a lightweight process that runs within a process.

Q.(73) Explain different ways of creating a thread.

Answer-- There are two ways to create a thread in Java:

By extending the Thread class and overriding the run() method.

By implementing the Runnable interface and passing an instance of the class to the Thread constructor.

Q.(74) What are different states in lifecycle of Thread?

Answer-- The different states in the lifecycle of a thread are:

New: A thread is in the new state when it is created but not started.

Runnable: A thread is in the runnable state when it is started and ready to run.

Running: A thread is in the running state when it is currently executing.

Blocked: A thread is in the blocked state when it is waiting for a resource or a synchronization object.

Terminated: A thread is in the terminated state when it has completed execution or has been stopped.

Q.(75) What is the difference between a synchronized method and a synchronized block?

Answer-- A synchronized method is a method that is synchronized on the this object.

A synchronized block is a block of code that is synchronized on a synchronization object.

Q.(76) What is difference between user Thread and daemon Thread?

Answer-- A user thread is a thread that is created by the application and has a higher priority than a daemon thread.

A daemon thread is a thread that is created by the JVM and has a lower priority than a user thread.

Q.(77) What is difference between stop() and suspend()?

Answer-- The stop() method is used to stop a thread immediately. It is deprecated and should not be used.

The suspend() method is used to suspend a thread temporarily. It is also deprecated and should not be used.

Q.(78) What is difference between join() and sleep()?

Answer-- The join() method is used to wait for a thread to complete execution.

The sleep() method is used to pause a thread for a specified amount of time.

Q. (79) What is difference between wait() and join()?

Answer-- The wait() method is used to wait for a specified amount of time or until a condition is met.

The join() method is used to wait for a thread to complete execution.

Q. (80) What is Java?

Answer-- Java is a high-level, object-oriented programming language developed by Sun Microsystems (now owned by Oracle). It is platform-independent, meaning once compiled, Java code can run on any device that supports Java without recompilation.

Q. (81) What are the main features of Java?

Answer-- Some of the main features of Java include:

Platform independence

Object-oriented

Simple

Secure

Portable

Robust

Multithreaded

Dynamic

Q. (82) What is the difference between JDK, JRE, and JVM?

Answer-- JDK (Java Development Kit) is a software development kit used to develop Java applications. It includes tools such as the Java compiler, Javadoc, and Java debugger.

JRE (Java Runtime Environment) is a runtime environment that is needed to run Java applications. It includes JVM, class libraries, and other supporting files.

JVM (Java Virtual Machine) is an abstract machine that provides a runtime environment in which Java bytecode can be executed.

Q. (83) What is the main difference between == and .equals() method in Java?

Answer-- The == operator is used to compare object references, i.e., whether two references point to the same memory location.

The .equals() method is used to compare the contents of two objects. It is usually overridden by classes to provide meaningful comparison.

Q. (84) What is OOP (Object-Oriented Programming) and how does Java support it?

Answer-- OOP is a programming paradigm based on the concept of "objects", which can contain data, in the form of fields, and code, in the form of procedures. Java supports OOP through features like classes, objects, encapsulation, inheritance, and polymorphism.

Q. (85) What is the difference between an abstract class and an interface in Java?

Answer-- An abstract class can have both abstract and concrete methods, whereas an interface can only have abstract methods (prior to Java 8).

A class can implement multiple interfaces but can extend only one abstract class.

Abstract classes can have constructors, while interfaces cannot.

Q. (86) What is the difference between method overloading and method overriding?

Answer-- Method overloading occurs when two or more methods in the same class have the same method name but different parameters.

Method overriding occurs when a subclass provides a specific implementation of a method that is already provided by its superclass.

Q. (87) What is the purpose of the "static" keyword in Java?

Answer-- The static keyword is used to create variables and methods that belong to the class rather than to instances of the class. Static members are shared among all instances of the class.

Q. (88) What is the use of the "final" keyword in Java?

Answer-- The final keyword is used to restrict the user from modifying variables, methods, and classes. When a variable is declared as final, its value cannot be changed. When a method is declared as final, it cannot be overridden. When a class is declared as final, it cannot be subclassed.

Q. (89) What is a constructor in Java?

Answer-- A constructor is a special type of method that is used to initialize objects. It is called when an object of a class is created. Constructors have the same name as the class and do not have a return type.